

CLI 工具自动 MCP 化与技能生成

CLI 工具自动 MCP 化与技能生成：方案准备学习笔记

1. 从 LLM Agent 说起：为什么 agent 需要工具
2. 工具调用的基本模式：ReAct、function calling 与 tool use
3. CLI 工具为什么适合作为研究对象
4. MCP tools schema：把工具调用变成结构化接口
5. Skill 文件：补足 schema 之外的使用经验
6. 从 CLI 文档生成 schema 和 skill：信息抽取的核心问题
7. 沙箱执行：真实调用工具，同时控制风险
8. 自动评测：任务成功率、token 消耗、对照实验
9. 失败归因与反馈修正
10. 对本项目的启发：最小可行系统应包含什么
 - 10.1 CLI 候选工具选择
 - 10.2 文档采集器
 - 10.3 schema 与 skill 生成器
 - 10.4 schema 合法性校验
 - 10.5 沙箱执行器
 - 10.6 agent 执行循环
 - 10.7 任务集与判分器
 - 10.8 实验记录与结果表格
 - 10.9 失败案例分析模板

小结

常见误区

后续阅读路线

参考资料

CLI 工具自动 MCP 化与技能生成：方案准备学习笔记

日期：2026-07-28

本文服务于浙大控制学院夏令营考核题目 2：“CLI 工具的自动 MCP 化与技能生成”。目标不是马上确定最终实现，而是在写方案和代码之前，先建立一条清晰的概念链路：

LLM Agent -> 工具调用 -> CLI 工具 -> MCP tools schema -> skill 文件
-> 文档到 schema/skill 的生成 -> 沙箱执行 -> 自动评测
-> 失败归因与反馈修正 -> 本项目最小可行系统

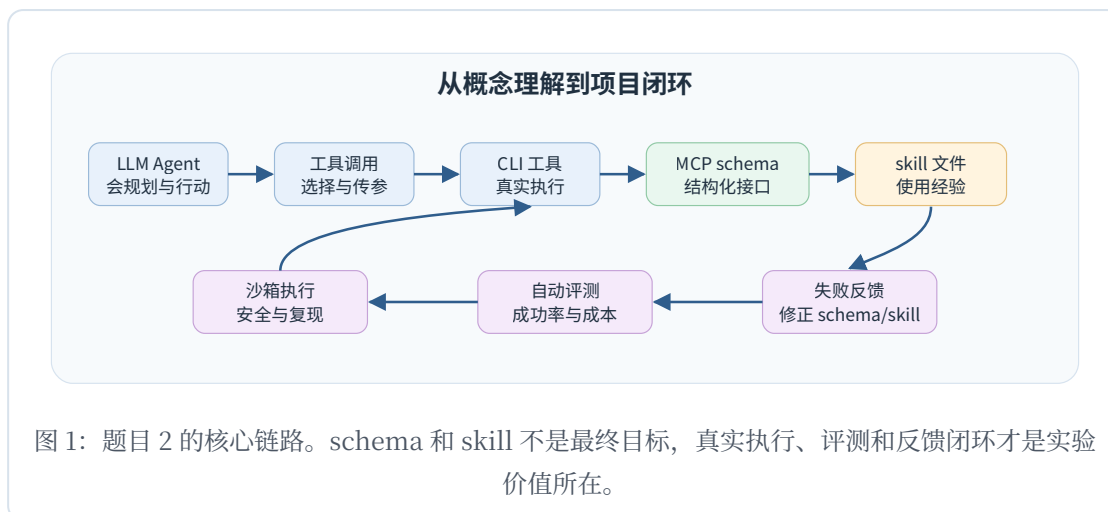
读完本文后，应能理解题目 2 到底在研究什么，以及后续项目为什么需要 pipeline、schema 校验、skill 说明、沙箱、任务集、自动判分和失败分析。

本文可以按两遍阅读。第一遍只抓住主线：agent 为什么需要工具，MCP schema 和 skill 分别解决什么，为什么必须真实执行和自动评测。第二遍再关注项目设计启发：哪些信息要从 CLI 文档中抽取，哪些风险要在沙箱里控制，哪些指标可以证明生成结果有用。

为了避免概念混在一起，先给出几个工作定义：

- **工具**：模型可以请求外部系统执行的能力，例如搜索文件、解析 JSON、调用 API、转换图片。
- **工具描述**：告诉模型工具叫什么、能做什么、参数是什么的说明。
- **工具调用**：模型决定使用某个工具，并生成一组参数。
- **执行环境**：真正运行工具的宿主系统，可以是本机、容器、远程服务或受限沙箱。
- **skill**：给模型看的可复用操作经验，强调使用场景、步骤、约束和常见错误。

- **评测任务**：用来检验 agent 是否真的能完成工作的输入、环境和判分标准。



1. 从 LLM Agent 说起：为什么 agent 需要工具

普通 LLM 的基本能力是根据上下文生成文本。它可以解释概念、写代码、总结材料，但它本身并不会天然拥有实时数据库、文件系统、命令行程序、网页、企业系统或实验环境的真实访问能力。

Agent 的关键变化是：模型不只是“回答”，而是在一个循环中行动。典型过程包括：

- 观察用户任务和当前环境；
- 判断是否需要外部信息或外部操作；
- 选择一个工具；
- 生成工具参数；
- 接收工具执行结果；
- 根据结果继续推理、修正或完成任务。

因此，agent 能力的上限不只取决于模型参数，也取决于它能否正确使用外部工具。一个没有工具的模型，面对“统计这个目录下所有 JSON 文件里的字段分布”时只能给出建议；一个能调用文件系统、jq、rg 或 Python 的 agent，则可以真实完成任务并返回结果。

这正是题目 2 的出发点：如果 agent 要使用新工具，它需要知道工具“能做什么、怎么调用、什么时候该调用、调用失败后怎么办”。目前这些信息往往由人工写 MCP 工具描述、JSON Schema 或 skill 文件。题目 2 希望自动化这条链路。

这里要区分“模型会说”和“agent 会做”。例如模型可能知道 `jq` 是处理 JSON 的工具，也能写出看似合理的 `jq '.items[]' file.json`。但在真实任务里，它还需要知道文件路径在哪里、JSON 结构是什么、输出应该保存到哪里、命令失败后该看 `stdout` 还是 `stderr`、能不能覆盖已有文件。只知道工具名和大概用途，并不等于能稳定完成任务。

另一个容易混淆的点是 agent 与自动化脚本的关系。传统脚本的流程由人提前写死，适合稳定重复任务；agent 则把部分决策交给模型，适合任务描述自然语言化、步骤不完全固定、需要根据中间结果调整的场景。题目 2 关注的不是替代所有脚本，而是让 agent 面对一个新 CLI 工具时，能更快、更稳地获得“可执行知识”。

因此，后续设计中需要同时照顾两类可靠性：

- **接口可靠性**：模型生成的参数能被机器校验，减少拼写、类型和必填项错误。
- **行为可靠性**：模型知道在什么上下文里使用工具，能避开危险操作，并能根据失败结果修正。

2. 工具调用的基本模式：ReAct、function calling 与 tool use

理解 agent 工具调用，可以先看 ReAct。ReAct 的核心思想是把“推理”和“行动”交错起来：模型不是先在脑中想完所有步骤再一次性输出答案，而是在任务过程中一边推理、一边调用外部环境，再根据观察结果调整后续动作。ReAct 论文强调，行动让模型能接触外部知识或环境，推理则帮助模型规划、跟踪进度和处理异常。见 ReAct: Synergizing Reasoning and Acting in Language Models。

在工程系统里，这种模式通常表现为 tool calling 或 function calling。官方 OpenAI function calling 文档把工具调用拆成几个概念：应用向模型提供可用工具定义；模型在需要时返回 tool call；应用执行工具；应用再把 tool output 传回模型，模型据此生成最终回答。见 OpenAI Function Calling。

这些机制说明一个事实：工具调用不是“模型直接执行代码”，而是模型、宿主应用和工具运行环境之间的一种协议。模型负责选择工具和生成参数，宿主应用负责验证、执行和返回结果。

这对本项目有两个启发：

- 工具接口必须结构清楚，否则模型很难稳定生成正确参数；
- 执行结果必须可观察、可记录，否则无法做自动评测和失败归因。

可以把一次工具调用拆成五个阶段：

1. **工具发现**：模型知道当前有哪些工具可用。
2. **工具选择**：模型判断哪一个工具最适合当前任务。
3. **参数构造**：模型为工具填写结构化参数。
4. **外部执行**：宿主系统运行工具，并捕获结果。
5. **结果利用**：模型读取工具输出，继续下一步或给出答案。

每个阶段都可能失败。工具发现阶段可能因为描述太模糊而漏选；工具选择阶段可能因为多个工具用途相似而选错；参数构造阶段可能因为 schema 不完整而填错类型；外部执行阶段可能因为环境依赖或权限不足而失败；结果利用阶段可能因为模型误读输出而给出错误结论。

ReAct 的价值在于，它让失败有机会被中间观察暴露出来。模型不必一次性猜完整答案，而可以在每一步看到环境反馈。题目 2 的 schema/skill 生成也应服务于这种闭环：schema 让动作更可执行，skill 让模型更会选择动作和解释观察。

function calling 与 MCP 都可以看成工具协议的一种实现方式。function calling 更像应用内部定义函数给模型用；MCP 更强调统一协议，让不同客户端和服务端之间能以标准方式发现和调用工具。对于本项目来说，不必一开始纠结两者的全部生态差异，关键是理解共同点：工具能力需要被结构化描述，模型输出的调用请求需要被宿主系统验证和执行。

3. CLI 工具为什么适合作为研究对象

题目 2 选择 CLI 工具，而不是任意 API 或 GUI 软件，是有现实原因的。

CLI 工具通常具备几个优势：

- 文档容易获取：大多数 CLI 有 `--help`、man page、README 或 examples。
- 输入输出可文本化：参数、stdout、stderr、退出码都容易记录。
- 任务容易构造：可以在临时目录里准备文件，让工具处理后检查结果。
- 自动判分可行：输出文件、文本内容、JSON 字段、退出码都可以作为成功标准。
- 适合沙箱隔离：工具可以在受限目录、受限命令和受限时间内运行。

但 CLI 工具也有天然难点：

- 子命令多，例如 `git commit`、`git log`、`git diff` 的参数体系完全不同；
- 参数关系复杂，例如互斥参数、默认值、短参数和长参数；
- 文档不一定机器友好，`--help` 往往面向人类阅读；
- 一些命令有破坏性，例如删除文件、改写仓库、访问网络；
- 错误输出可能含混，需要经验才能定位原因。

因此，从 CLI 文档直接让模型“自己看懂并使用”，并不稳定。题目 2 的价值就在于：能否自动把人类文档转成 agent 更容易使用的结构化接口和操作经验。

CLI 工具还有一个适合教学和实验的优点：它天然带有“可观察副作用”。比如 `rg` 的结果是命中的文本行，`jq` 的结果是 JSON 变换后的 stdout，`git status --short` 的结果是仓库状态，`convert` 或 `ffmpeg` 的结果是生成文件。这些结果不像开放式问答那样难判定，比较容易写成自动检查。

但也正因为 CLI 能改文件、删文件、联网、启动进程，不能把所有命令都直接暴露给 agent。题目 2 的候选工具选择应偏向：

- 能在临时目录中完成任务；
- 常见操作不需要管理员权限；

- 输出稳定，容易判分；
- 危险能力可以通过 allowlist 禁掉；
- 文档中有足够示例，便于抽取经验。

举一个对比：jq 比较适合早期实验，因为它输入 JSON、输出 JSON 或文本，判分简单；git 也有价值，但最好先限制在只读子命令，如 status、log、diff、show，避免一开始让 agent 修改历史或提交。这样的工具选择不是工程细节，而是实验可控性的前提。

从研究角度看，CLI 工具也能代表一类更广泛的问题：许多真实软件的文档都是给人读的，而 agent 需要的是可调用、可验证、可恢复的操作知识。CLI 只是这个问题的一个容易落地的切入点。

4. MCP tools schema：把工具调用变成结构化接口

MCP，即 Model Context Protocol，是一种让模型宿主连接外部工具和数据源的协议。MCP tools 规范说明：服务器可以暴露能被语言模型调用的工具；工具用名称唯一标识，并带有描述和 schema。见 MCP Tools Specification。

在 MCP tools 里，一个工具定义通常包含：

- name：工具唯一名称；
- title：可选的人类可读名称；
- description：工具功能说明；
- inputSchema：输入参数的 JSON Schema；
- outputSchema：可选的输出结构；
- annotations：可选的工具行为标注；
- execution：可选的执行相关属性。

其中 inputSchema 是本项目最关键的部分。它把“工具参数应该长什么样”变成机器可读格式。例如参数是字符串、数字、布尔值还是数组，哪些字段必填，哪些字段不能额外出现，都可以通过 JSON Schema 表达。

这解决的是“怎么调用”的问题。没有 schema 时，模型看到的是一段 `--help` 文本；有 schema 后，模型面对的是明确的参数槽位和类型约束。宿主系统也可以在执行前检查参数是否符合 schema，从而减少无意义调用。

但 schema 不解决全部问题。它通常不能充分表达：

- 什么任务场景下应该调用这个工具；
- 多个参数怎样组合更可靠；
- 哪些子命令适合作为高频能力暴露；
- 哪些危险参数应该禁用；
- 工具失败后如何修正；
- 多个工具如何组合成 workflow。

所以，schema 是必要的接口层，但不是完整的工具学习。

一个简化的 MCP tool schema 可以这样理解：

```
{
  "name": "jq_query",
  "description": "Run a jq filter on a JSON file and return the output.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "file": { "type": "string" },
      "filter": { "type": "string" }
    },
    "required": ["file", "filter"],
    "additionalProperties": false
  }
}
```

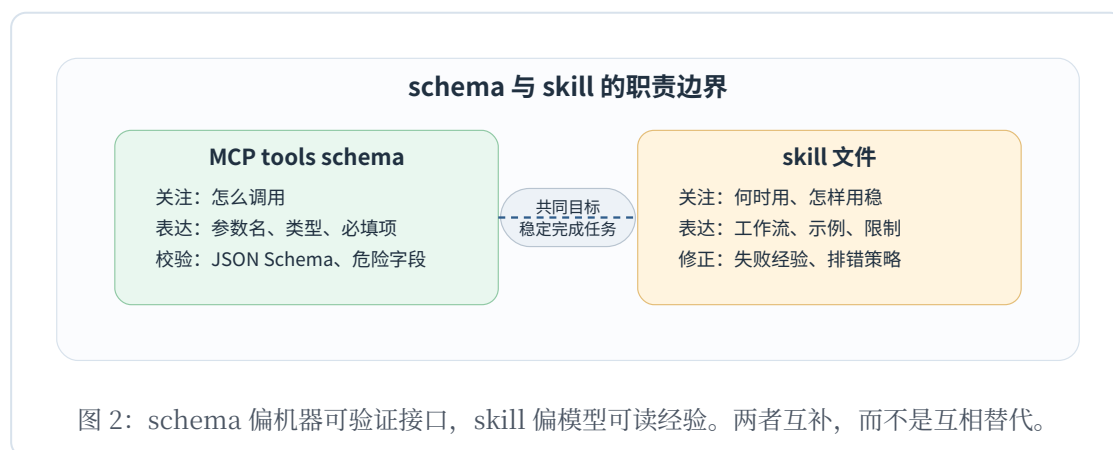
这个 schema 能约束“必须提供文件路径和 jq filter，不能多传未知字段”。但它仍然没有告诉模型：filter 应该怎样写、什么时候需要 `-r`、输入文件不存在怎么办、输出要不要重定向、处理数组和对象时有哪些常见模式。这些都属于 schema 之外的使用知识。

设计 schema 时要避免两个极端：

- **过宽**：把参数写成一个自由字符串 `command`，模型可以填任何 shell 命令。这样灵活，但几乎不可控，也很难校验安全性。
- **过窄**：只暴露一个固定场景，例如只能读取某个字段。这样安全，但泛化能力弱，无法覆盖真实任务。

较好的做法通常是把 CLI 的能力切成若干安全、清晰、高频的工具。比如不要直接暴露完整 `git`，而是暴露 `git_status`、`git_show_file`、`git_log_recent` 这类边界更清楚的工具。对于题目 2，这个切分策略本身就是值得分析的设计点：自动生成 pipeline 不仅要抽参数，还要判断哪些命令适合作为 agent 工具。

MCP 规范还提醒了一个安全事实：工具通常由模型控制调用，但应用应让用户理解哪些工具暴露给模型，并在敏感操作上保留人工确认。这说明本项目的 schema 生成不能只追求“能调通”，还要考虑权限、危险操作和可解释性。



5. Skill 文件：补足 schema 之外的使用经验

Skill 可以理解为给 agent 的“可复用操作说明”。Anthropic 的 Agent Skills 文档把 skill 描述为基于文件系统的资源，用来提供特定领域的工作流、上下文和最佳实践；它们按需加载，避免每次对话都重复塞入完整说明。见 Claude Agent Skills Overview。

一个典型 skill 至少包含元数据和正文说明：

- 元数据告诉 agent：这个 skill 是什么，什么时候应该用；
- 正文说明告诉 agent：具体工作流、常用方法、注意事项、示例；

- 额外资源可以提供参考文档、脚本、模板或测试数据。

这与 MCP schema 形成互补：

- schema 像接口签名，强调参数结构和可验证性；
- skill 像操作手册，强调使用策略、经验和失败处理。

例如，对于一个图像转换 CLI，schema 可以说明参数 `input_path`、`output_path`、`format` 的类型；skill 则应该说明“先检查输入文件是否存在”“批量处理时保留原目录结构”“转换失败时查看 `stderr` 中的 `codec` 信息”“不要覆盖原文件，除非任务明确要求”。

题目 2 要生成的不只是 MCP schema，还包括 skill 文件。原因很直接：如果只生成 schema，模型可能会“会填参数”，但仍然不知道什么时候应该用、怎么组合调用、如何规避常见错误。

Voyager 也能帮助理解 skill 的意义。Voyager 在 Minecraft 环境中维护一个不断增长的技能库，把可执行代码形式的复杂行为保存和复用，并结合环境反馈、执行错误和自验证改进程序。见 Voyager: An Open-Ended Embodied Agent with Large Language Models。虽然 Voyager 的场景不是 CLI 工具，但它说明了一个共同思想：agent 不应每次从零开始解决任务，而应沉淀可复用、可组合、可检索的经验。

Skill 的价值还体现在“渐进加载”。如果把所有工具文档都直接塞进 prompt，模型会消耗大量上下文，而且容易被无关细节干扰。Skill 的思路是先用简短元数据做检索或触发，只有当任务相关时再加载更详细的说明。这对题目 2 很重要，因为 CLI 文档可能很长，而任务通常只需要其中一小部分。

可以把 skill 文件写成几类信息：

- **适用场景**：什么任务应该用这个工具，什么任务不应该用。
- **快速路径**：最常见的 2 到 3 种调用方式。
- **参数经验**：常见参数组合、默认值、路径约定。
- **安全规则**：禁止删除、覆盖、联网或写出沙箱目录。
- **失败处理**：遇到退出码非零、空输出、格式错误时先检查什么。
- **组合方式**：这个工具如何与其他工具配合，例如 `rg` 找文件后用 `jq` 处理 JSON。

例如针对 `rg`, `schema` 可能只说明 `pattern`、`path`、`case_sensitive` 等参数。`skill` 则应该提醒：如果用户要找文件名而不是文件内容，应使用文件列表能力而不是全文搜索；如果结果太多，应先缩小目录；如果要机器读取结果，应优先选择稳定格式，而不是依赖彩色终端输出。

因此，本项目的生成质量不能只看 `schema` 是否合法。一个合法但没有经验的 `schema`，可能仍然让 `agent` 反复试错；一个写得好的 `skill`，则能减少无意义调用，降低 `token` 消耗，并让失败更容易恢复。

6. 从 CLI 文档生成 `schema` 和 `skill`：信息抽取的核心问题

从 CLI 文档到 `schema/skill`，本质上是一个文档理解和结构化生成问题。输入可能包括：

- `tool --help`；
- `tool subcommand --help`；
- README；
- 官方手册；
- 示例命令；
- 错误信息说明。

生成 MCP `schema` 时，至少需要抽取：

- 命令和子命令名称；
- 参数名称，包括短参数和长参数；
- 参数类型，例如 `string`、`number`、`boolean`、`array`、`enum`；
- 必填参数和可选参数；
- 默认值；
- 参数互斥关系；
- 参数依赖关系；
- 输入文件和输出文件；
- 退出码与错误含义。

生成 skill 时，则更关注策略信息：

- 这个工具适合解决哪些任务；
- 典型调用顺序是什么；
- 哪些参数组合最常用；
- 什么时候应该先检查文件或环境；
- 有哪些危险操作必须避免；
- 失败后优先检查什么；
- 多工具任务中它应该放在哪一步。

已有 CLI-to-MCP 项目说明这个方向已经有现实需求。例如 any-cli-mcp-server 的 README 表示它可以使用 CLI 的 `--help` 输出构建 MCP tools，并支持 GitHub CLI、Azure CLI、Git 等工具。这类项目对本项目有参考意义：它证明“从 CLI help 构造 MCP 接口”可行；同时也提示我们，夏令营题目可以在“生成质量、skill 说明、任务成功率、失败反馈”上继续做更系统的实验。

本项目后续可以把生成 pipeline 拆成几个阶段：

1. 文档采集：运行 `--help` 或读取 README。
2. 候选能力识别：判断哪些子命令值得暴露成工具。
3. 参数抽取：生成结构化参数草稿。
4. schema 生成：输出 MCP tool schema。
5. skill 生成：输出自然语言使用说明。
6. 静态校验：检查 JSON Schema 合法性和字段完整性。
7. 动态验证：让 agent 用它完成任务。
8. 失败回传：用失败日志修正 schema 或 skill。

这条 pipeline 的难点不是“把文本改成 JSON”，而是如何让生成结果真的提高 agent 的任务成功率。

从 CLI 文档抽取信息时，需要注意文档里经常有隐含结构。比如：

- **Usage**：行通常暗示位置参数和可选参数；
- **Options**：列表通常暗示短参数、长参数和布尔开关；

- 示例命令往往比参数列表更能说明真实用法；
- “default” “required” “cannot be used with” 等文本暗示约束；
- stderr 中的错误信息能反向说明参数限制；
- README 的教程可能暴露 `--help` 中没有的最佳实践。

如果只解析 `--help`，生成速度快，但可能漏掉语义；如果同时读 README 和手册，信息更充分，但噪声更多，token 成本也更高。后续方案可以把文档来源分层：先用 `--help` 生成基础 schema，再用 README 示例补充 skill，而不是一开始把所有文档混在一起处理。

生成过程还需要保留“证据链”。每个 schema 字段最好能追溯到原始文档中的某个片段，例如参数名称来自哪一行、类型为什么判断为 `boolean`、某个 `enum` 值来自哪个示例。这样做有两个好处：

- 人工审查时可以快速发现误抽取；
- 失败反馈时可以判断是原始文档缺失，还是生成器误读。

还要区分“抽取”和“设计”。抽取是从文档中识别已有参数；设计是决定如何把原始 CLI 能力包装成 agent 友好的工具。比如原始 CLI 可能只有一个 `tool` 命令，接受自由参数；但 MCP 层可以拆成多个更安全的工具。题目 2 的创新空间，往往就在抽取之后的设计和验证环节。

一个合理的生成结果应至少通过三类检查：

1. **格式检查**：JSON 可解析，schema 合法，字段名符合约定。
2. **静态语义检查**：必填项、类型、默认值、危险参数没有明显问题。
3. **动态任务检查**：agent 使用这些产物完成任务的成功率确实提升。

前两类检查证明“产物看起来能用”；第三类检查才证明“产物真的有用”。

7. 沙箱执行：真实调用工具，同时控制风险

题目 2 要求在容器化沙箱环境中搭建最小 agent 执行循环。这一点非常重要，因为仅靠人工阅读 schema/skill 无法证明它们有用。

真正的评测应该让 agent 在受控环境中执行任务。例如：

- 给定一个临时目录；
- 放入若干输入文件；
- 给 agent 暴露一组工具；
- agent 根据任务调用工具；
- 工具真实运行；
- 系统记录命令、参数、stdout、stderr、退出码和产物；
- 判分器检查最终状态。

沙箱的作用是控制风险和保证复现：

- 文件操作限制在任务临时目录；
- 任务结束后销毁环境；
- 禁止危险命令或危险参数；
- 设置超时，避免无限运行；
- 必要时禁止网络；
- 每次任务从干净初始状态开始。

这也解释了为什么 CLI 工具适合该题。与远程 API 相比，CLI 任务可以用本地文件构造，成本低、重复性强、结果容易判定。

沙箱不一定一开始就做得很复杂。学习阶段只需要理解它在实验中的角色：把“看起来会用工具”变成“真实完成任务”，并把过程记录下来用于分析。

沙箱设计可以从威胁模型理解。agent 调用 CLI 时，主要风险包括：

- **文件越界**：读取或写入任务目录之外的文件。
- **破坏性操作**：删除输入、覆盖关键文件、修改 git 历史。
- **资源耗尽**：命令卡住、输出过大、递归扫描整个磁盘。
- **网络访问**：下载不受控内容或泄露环境信息。
- **命令注入**：把用户输入拼进 shell 字符串后产生额外命令。
- **不可复现**：任务依赖当前机器状态，换环境就失败。

为了降低这些风险，沙箱至少需要几类边界：

- 目录边界：每个任务只看到自己的临时目录。
- 命令边界：只允许 allowlist 中的工具和子命令。
- 参数边界：禁止危险参数，路径必须规范化后检查。
- 时间边界：每次调用设置超时。
- 输出边界：限制 stdout/stderr 长度，避免日志爆炸。
- 环境边界：清理环境变量，禁用不必要网络。

这也影响 schema 设计。比如如果 schema 允许传入任意 `command` 字符串，沙箱就必须承担极高的解析和拦截压力；如果 schema 把参数拆成结构化字段，宿主系统就能在执行前检查路径、枚举值和危险开关。换句话说，好的 schema 本身就是沙箱安全的一部分。

实验记录也应由沙箱统一生成，而不是让模型自己描述。模型的总结可能遗漏错误细节；系统日志则能记录真实命令、退出码、耗时和文件变化。这些信息后续会用于失败归因。

8. 自动评测：任务成功率、token 消耗、对照实验

题目 2 的评价不是“生成的 schema 看起来合理”，而是“agent 使用这些生成结果后，任务是否更容易成功，token 是否更省”。

因此任务集必须有自动判定标准。常见判分方式包括：

- 检查某个输出文件是否存在；
- 检查文件内容是否包含目标字符串；
- 检查 JSON 字段和数值是否正确；
- 检查图像、音频或压缩包等产物属性；
- 检查命令退出码；
- 检查目录结构是否符合预期。

对照实验至少需要两组：

- 实验组：模型看到生成的 MCP schema 和 skill；

- 对照组：模型直接看到原始 CLI 文档。

如果实验组成功率更高，说明结构化接口和经验说明确实帮助了模型。如果实验组 token 更少，说明 schema/skill 可能减少了模型阅读长文档的成本。如果实验组失败率更高，则要分析是 schema 抽取错误、skill 误导模型，还是任务设计不合理。

ToolLLM 提供了一个相关参照。它围绕真实 API 构造 ToolBench，并考虑单工具和多工具场景、解决路径标注和自动评价。见 [ToolLLM](#)。虽然它关注 REST API 而不是 CLI，但它说明 tool-use 研究需要任务、工具、调用路径和评价器共同构成实验闭环。

Tool learning survey 进一步把工具学习流程概括为任务规划、工具选择、工具调用和响应生成等阶段，并总结了 benchmark 和评价方法。见 [Tool Learning with Large Language Models: A Survey](#)。这对本项目很有用：失败不一定发生在调用阶段，也可能发生在任务规划或工具选择阶段。

任务设计要避免两个问题：太简单和太开放。太简单的任务，比如“运行 `tool --version`”，很难体现 schema/skill 的价值；太开放的任务，比如“分析整个项目并提出改进”，又难以自动判分。更合适的是构造有明确输入、明确产物、但需要正确使用工具的任务。

可以设计一个难度梯度：

- **Level 1：单步参数任务。**例如用 `jq` 从 JSON 中取出字段，检查输出是否等于预期。
- **Level 2：单工具多步任务。**例如先查看 JSON 结构，再写过滤表达式，最后保存结果。
- **Level 3：双工具组合任务。**例如用 `rg` 找到包含目标键的文件，再用 `jq` 汇总字段。
- **Level 4：错误恢复任务。**输入文件中有一个格式错误，agent 需要根据错误信息定位并修正调用方式。
- **Level 5：约束敏感任务。**要求不覆盖原文件、不访问网络、不写出目录外。

除了成功率和 token 消耗，还可以记录辅助指标：

- 工具调用次数：调用越少，说明指导越有效。
- 首次成功率：第一次完整执行是否成功。
- 平均修正轮数：失败后需要几轮观察和重试。
- 无效调用比例：参数校验失败或被沙箱拒绝的调用占比。
- 失败类别分布：规划错、选工具错、参数错、环境错、判分错分别多少。

这些指标不一定全部写进最终基础目标，但在分析时很有用。比如两组成功率相同，但实验组工具调用次数更少、token 更少，也能说明 schema/skill 有效率收益。

对照组设计要公平。原始文档直接输入模型时，也应该给它同样的任务、同样的沙箱、同样的工具执行能力。两组的差别应尽量集中在“工具知识表达形式”上，而不是模型、任务或运行环境不同。否则结果很难解释。

9. 失败归因与反馈修正

题目 2 的进阶目标要求把失败反馈回传给生成 pipeline，自动修正 schema 或 skill。要做到这一点，首先要能定位失败来自哪里。

一条失败链路可以这样回溯：

原始文档

- > 生成的 schema
- > 生成的 skill
- > 模型任务理解
- > 工具选择
- > 参数生成
- > 工具执行
- > 产物检查
- > 判分结果

常见失败类型包括：

- 文档本身缺信息：`--help` 没写清楚参数取值或示例；

- 抽取错误：把可选参数当成必填，把字符串当成布尔；
- schema 表达不完整：没有表达互斥参数或 enum；
- skill 说明缺关键经验：没有提示先检查输入、不要覆盖原文件；
- 模型选错工具：任务需要 jq，模型却尝试用 rg；
- 参数组合错误：路径、格式、过滤条件不匹配；
- 沙箱限制导致失败：工具需要网络或写出目录外文件；
- 判分器过严或过松：任务其实完成了但判分失败，或错误产物被判成功。

反馈修正循环可以从简单版本开始：

1. 记录失败日志。
2. 判断失败类别。
3. 把失败信息输入生成器。
4. 只允许修正 schema 或 skill 中相关部分。
5. 重新运行相同任务。
6. 比较修正前后的成功率和 token 消耗。

这里的关键是“闭环”。如果系统只能生成一次 schema/skill，研究价值有限；如果能从失败中改进，就更接近题目要求中的“试错-改进”机制。

失败归因可以设计成半结构化记录，而不是只写自然语言总结。例如每个失败任务记录：

```
task_id: jq_03
stage: parameter_generation
symptom: jq returned compile error
evidence: stderr contains "syntax error"
root_cause: skill did not explain array indexing syntax
repair_target: skill
repair_action: add example for extracting array fields
rerun_result: success
```

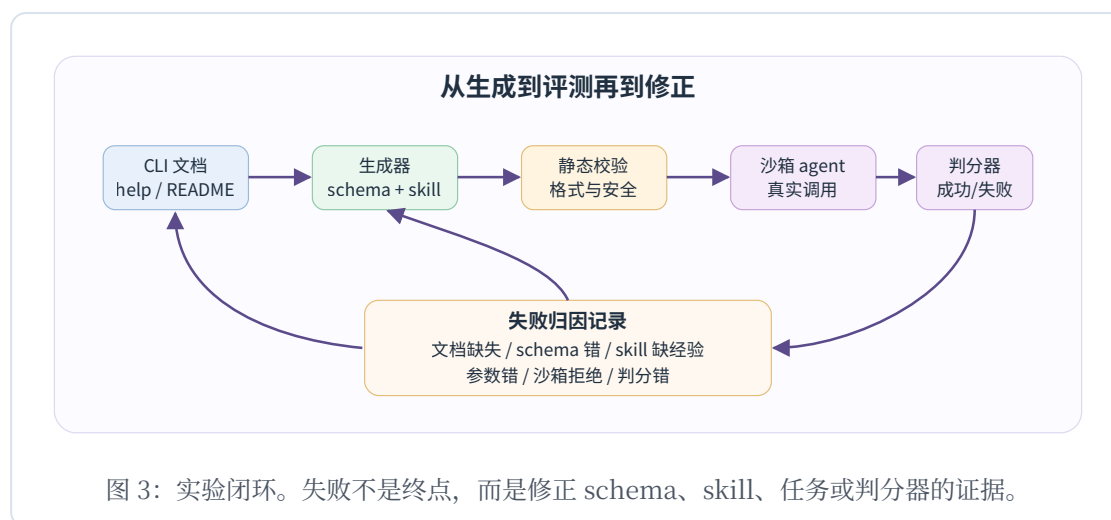
这种记录方式能直接服务于报告。它让读者看到失败不是被简单归为“模型不行”，而是能沿链路定位到具体环节。对于题目 2，这一点尤其重要，因为研究对象不是单个模型，而是“文档 -> schema/skill -> agent 执行”的整体系统。

反馈修正也要控制范围。不能每次失败后任意重写全部产物，否则很难知道改进来自哪里。更稳妥的策略是：

- 如果 JSON 解析失败，只修 schema 格式；
- 如果参数类型错，只修对应字段；
- 如果模型不知道何时使用工具，只修 skill 的适用场景；
- 如果模型反复危险调用，只修安全规则或 allowlist；
- 如果判分器误判，只修任务定义和判分器。

这样才能形成可解释的迭代实验。进阶目标中的“收敛速度”也可以由此定义：同一批任务经过几轮反馈后成功率趋于稳定，或者失败类型不再集中在 schema/skill 问题上。

还要警惕一种假改进：把任务答案直接写进 skill。这样会提高特定任务成功率，但破坏泛化。合理的修正应该增加工具使用知识，而不是泄露任务答案。



10. 对本项目的启发：最小可行系统应包含什么

经过上面的概念链路，题目 2 可以被理解为一个实验系统，而不是单个脚本。后续正式方案大概率需要包含以下模块。

10.1 CLI 候选工具选择

候选工具应满足：

- 本地可运行；
- 文档可获得；
- 有清晰输入输出；
- 容易构造自动判分任务；
- 危险操作可禁用；
- 能覆盖单步和多步任务。

初期可以优先考虑文本和结构化数据工具，例如 `rg`、`jq`、`git` 的只读子集，或其他输出稳定的 CLI。

候选工具可以按三类组合，而不是只选同质工具：

- **检索类**：如 `rg`，适合测试路径、模式匹配、结果筛选。
- **结构化处理类**：如 `jq`，适合测试参数表达、输出判分和错误恢复。
- **状态查询类**：如 `git` 的只读子命令，适合测试多子命令和上下文理解。

这样任务集能覆盖“找信息、处理信息、解释状态”三种常见 agent 工作，而不是只验证单一能力。

10.2 文档采集器

采集器负责收集 `--help`、子命令 `help`、README 和示例。它的输出应保存下来，作为后续生成和失败分析的证据。

文档采集阶段最好保存原始文本、来源路径、采集时间和命令退出码。比如 `jq --help` 和 `jq --version` 都应该留档。这样后续如果 schema 字段有争议，可以回到原始证据，而不是只看生成结果。

10.3 schema 与 skill 生成器

生成器负责从文档中生成两类产物：

- MCP tool schema：结构化接口；
- skill 文件：自然语言使用说明。

后续可以先人工审查，再逐步提高自动化程度。

生成器不一定一开始追求完全自动。基础版本可以采用“LLM 生成 + 程序校验 + 人工抽查”的方式。夏令营考核更看重实验链路是否清楚，而不是一开始就把所有判断都交给模型。

10.4 schema 合法性校验

校验器至少检查：

- JSON 是否可解析；
- `inputSchema` 是否是合法 JSON Schema 对象；
- 必填字段是否存在；
- 参数类型是否合理；
- 是否出现明显危险参数。

如果校验失败，应把错误反馈写成结构化信息，而不是只打印栈追踪。生成器后续可以利用这些错误修正 `schema`。例如“字段 `recursive` 的类型应为 `boolean`，但生成结果是 `string`”比“`schema invalid`”更有用。

10.5 沙箱执行器

沙箱执行器负责创建临时任务目录、复制输入文件、暴露允许工具、执行 agent 调用并记录日志。基础版本可以先关注文件系统隔离和超时控制。

沙箱执行器还应尽量让任务可重复。每次运行前重新创建目录，复制同一份输入，运行后保存输出快照。否则一次失败可能污染下一次实验，使结果不可解释。

10.6 agent 执行循环

最小 agent loop 不需要一开始追求复杂。它只要能：

- 接收任务；
- 读取工具描述；
- 选择工具；
- 生成参数；

- 执行工具；
- 观察结果；
- 必要时继续下一步；
- 输出最终答案或产物。

这个循环的重点不是让模型“像人一样自由使用终端”，而是让它在受控工具集合里做决策。越早限制好工具边界，后续数据越容易分析。

10.7 任务集与判分器

任务集需要覆盖难度梯度：

- 单工具单步任务；
- 单工具多步任务；
- 两个工具组合任务；
- 容易误用参数的任务；
- 需要从错误中恢复的任务。

每个任务都必须有自动判分函数，避免只靠人工判断。

任务描述也要避免给出过多工具暗示。例如“用 jq 提取字段”会直接泄露工具选择；“从 orders.json 中生成每个用户的订单总额”更适合作为 agent 任务。这样才能测到工具选择和使用说明是否有效。

10.8 实验记录与结果表格

每次运行至少记录：

- 任务 ID；
- 工具集合；
- 实验组或对照组；
- 模型；
- prompt token、completion token 和总 token；
- 工具调用次数；
- 是否成功；
- 失败类别；

- 关键日志路径。

如果预算允许，还可以记录每次工具调用前后的上下文长度。这样能更直接分析 skill 是否减少了模型阅读文档的 token 成本。

10.9 失败案例分析模板

报告中最好选 2 到 3 个典型失败案例，沿“文档 -> schema -> skill -> 决策 -> 执行 -> 判分”回溯。这会比只给一张成功率表格更有说服力。

失败案例最好覆盖不同类型。例如一个 schema 抽取错误、一个 skill 缺经验、一个模型决策错误。这样报告能展示系统分析能力，而不是只挑最容易解释的失败。

小结

题目 2 的核心不是简单地“给 CLI 包一层 MCP”。它真正研究的是：如何把面向人类的工具文档，自动转化为 agent 可用、可验证、可复用的工具接口和使用经验，并用真实任务成功率证明这种转化有价值。

从概念上看：

- ReAct 说明 agent 需要在推理和行动之间循环；
- function/tool calling 说明工具调用需要结构化协议；
- MCP tools schema 提供“怎么调用”的接口层；
- skill 文件补充“什么时候调用、怎样调用更稳”的经验层；
- CLI 工具提供低成本、可复现、可自动判分的实验对象；
- 沙箱让真实执行变得可控；
- 自动评测和失败归因让生成结果可以被数据检验。

因此，后续正式方案设计应围绕一个最小闭环展开：

CLI 文档 -> schema/skill 生成 -> schema 校验 -> 沙箱 agent 执行
-> 自动判分 -> 结果统计 -> 失败归因 -> 生成结果修正

只要这个闭环跑通，就能支撑题目 2 的基础目标；在此基础上，再考虑反馈修正、长尾工具泛化和动态精简 skill 等进阶方向。

读完本文后，可以用下面几个问题检查是否已经具备进入方案设计的基础：

- 能否用一句话解释 MCP schema 和 skill 的区别？
- 能否说清为什么不能只把 CLI 文档直接塞给模型？
- 能否为一个 CLI 工具列出 3 个适合自动判分的任务？
- 能否说明沙箱至少要限制哪些东西？
- 能否区分工具选择错误、参数错误和判分器错误？
- 能否设计一个失败反馈如何修正 skill 的例子？

如果这些问题能回答清楚，下一阶段就可以从“学习领域”转入“设计可行方案”：选工具、定任务、画 pipeline、定义数据格式和评价指标。

常见误区

在进入正式方案设计前，有几个误区需要提前避开。

第一个误区是把 MCP 化理解成“把命令行字符串包成一个工具”。如果只是提供一个 `run_command`，让模型传入任意命令，那么它确实能调用 CLI，但没有真正完成结构化。这样的接口很难校验参数，也很难限制危险操作，失败后更难判断是模型错、命令错还是环境错。题目 2 更关心的是从文档中抽象出稳定、可验证、适合 agent 使用的接口。

第二个误区是认为 schema 越详细越好。schema 需要表达参数结构，但不应该把所有文档内容都硬塞进字段描述里。过长的 description 会增加 token 成本，也可能让模型抓不住重点。好的 schema 应该短而准，把结构说清楚；更长的使用经验放到 skill 里，并按任务需要加载。

第三个误区是认为 skill 就是 prompt 模板。skill 更像可维护的操作手册。它应该能随着失败案例不断改进，包含 workflow、示例、限制和排错策略。如果 skill 只是几句泛泛的“请正确使用工具”，它对 agent 的实际帮助会很弱。

第四个误区是只看最终答案，不看执行轨迹。工具学习实验必须记录中间过程，因为同一个失败结果可能有完全不同原因。比如没有输出文件，可能是工具没被调用，也可能是参数错，也可能是沙箱拒绝写入，还可能是判分器检查错了路径。没有日志就无法做可信分析。

第五个误区是任务设计过度贴合生成结果。如果任务全部围绕生成器最擅长的参数写，实验组自然会表现很好，但这不能说明方法有泛化价值。更合理的任务集应该覆盖常规任务、边界任务和容易误用的任务。

后续阅读路线

如果继续深入学习，可以按下面顺序读资料。

1. 先读 ReAct，理解 agent 为什么需要“推理、行动、观察”的闭环。
2. 再读 OpenAI function calling 或类似 tool calling 文档，理解工程系统中工具定义、工具调用和工具输出的基本流程。
3. 接着读 MCP tools 规范，重点看 `tools/list`、`tools/call`、`inputSchema`、`outputSchema` 和安全提示。
4. 然后读 Agent Skills 相关文档，理解 skill 为什么是按需加载的文件系统资源，而不只是长 prompt。
5. 再看 any-cli-mcp-server 这类项目，理解已有 CLI-to-MCP 做法能解决什么、还缺什么。
6. 最后读 ToolLLM 和 tool learning survey，把本项目放到工具学习评测的大背景中。

这条路线的目的不是读完所有论文细节，而是回答一个面向项目的问题：如何证明“生成的结构化接口和 skill”比“直接给原始文档”更能帮助 agent 完成真实任务。

参考资料

- 夏令营考核要求原文
- MCP Tools Specification, 2025-11-25
- Claude Agent Skills Overview

- OpenAI Function Calling
- ReAct: Synergizing Reasoning and Acting in Language Models
- Voyager: An Open-Ended Embodied Agent with Large Language Models
- ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs
- Tool Learning with Large Language Models: A Survey
- any-cli-mcp-server